Note de séance SQL n°4

Sous-requêtes et CTE | 14 et 15 octobre 2026

1. L'idée de la séance

Construire une requête **en plusieurs étapes**. Une **sous-requête** est une requête entre parenthèses dont le résultat sert à la requête principale. Un **CTE** (Common Table Expression, déclaré avec `WITH`) est une étape intermédiaire à laquelle on donne un nom, utilisable ensuite comme une vraie table. Les CTE demandent MySQL 8.0 ou plus.

2. Sous-requêtes

```
-- Clients plus âgés que la moyenne (33.25) : Jean, Patrick, Sarah, Eric
SELECT prenom, age FROM clients
WHERE age > (SELECT AVG(age) FROM clients);

-- Clients qui ont commandé (IN) : Awa, Jean, Marie, Grace
SELECT prenom FROM clients
WHERE id IN (SELECT client_id FROM commandes);

-- Clients sans commande (NOT IN) : Patrick, David, Sarah, Eric
SELECT prenom FROM clients
WHERE id NOT IN (SELECT client_id FROM commandes);

-- Commandes au-dessus de la moyenne : Telephone 250, Ordinateur 800, Tablette 300
SELECT * FROM commandes
WHERE montant > (SELECT AVG(montant) FROM commandes);

-- Le client le plus jeune : Marie (22)
SELECT * FROM clients
WHERE age = (SELECT MIN(age) FROM clients);

-- La commande la plus chère avec le prénom du client : Jean, 800
SELECT o.*, c.prenom
FROM commandes o
INNER JOIN clients c ON o.client_id = c.id
WHERE o.montant = (SELECT MAX(montant) FROM commandes);
```

3. Les CTE

```
-- Dépense par client, puis filtre : Awa 290, Jean 800, Grace 300
WITH depenses AS (
    SELECT client_id, SUM(montant) AS total
    FROM commandes
    GROUP BY client_id
)
SELECT c.prenom, d.total
FROM clients c
INNER JOIN depenses d ON c.id = d.client_id
WHERE d.total > 200;

-- Les clients de la ville qui en compte le plus (Kinshasa)
WITH parville AS (
    SELECT ville, COUNT(*) AS nb
```

```
    FROM clients
    GROUP BY ville
    ORDER BY nb DESC
    LIMIT 1
)
SELECT c.*
FROM clients c
INNER JOIN parville p ON c.ville = p.ville;
```

4. CTE enchaînés : comparer à la moyenne

Trois étapes : la dépense de chaque client, la moyenne de ces dépenses (358.75), puis la comparaison. Les CTE sont séparés par une **virgule**, et chacun peut utiliser ceux déclarés avant lui.

```
WITH depenses AS (
    SELECT c.id AS client_id, c.prenom, SUM(o.montant) AS total
    FROM clients c
    INNER JOIN commandes o ON c.id = o.client_id
    GROUP BY c.id, c.prenom
),
moyenne AS (
    SELECT AVG(total) AS moy FROM depenses
)
SELECT prenom, total
FROM depenses
WHERE total > (SELECT moy FROM moyenne);
-- Résultat : Jean (800)
-- Avec < : Awa 290, Grace 300, Marie 45
```

5. CROSS JOIN et GROUP BY multi-colonnes

• **CROSS JOIN** associe chaque ligne d'une table avec chaque ligne de l'autre, sans condition ON. Pratique pour coller une valeur unique (la moyenne) à côté de chaque ligne. Attention : le nombre de lignes est le produit des deux tailles.

```
SELECT d.prenom, d.total, m.moy
FROM depenses d
CROSS JOIN moyenne m
WHERE d.total > m.moy;
```

• **GROUP BY c.id, c.prenom** : toute colonne du SELECT doit être dans le GROUP BY ou dans une fonction d'agrégation. L'id sépare les personnes, le prénom permet de l'afficher. Sans clé primaire déclarée, MySQL ne devine pas que l'id est unique.

6. Comparaison des méthodes

Besoin	Méthode	Remarque
Comparer à une valeur unique	Sous-requête dans WHERE	Simple et lisible
Liste de valeurs	IN / NOT IN	NOT IN échoue si la liste contient NULL
Plusieurs étapes	CTE (WITH)	Plus lisible que des sous-requêtes imbriquées
Lignes sans correspondance	LEFT JOIN ... IS NULL	Plus sûr que NOT IN

7. À retenir

• Une sous-requête entre parenthèses renvoie soit une valeur (comparée avec >, <, =), soit une liste (utilisée avec IN).

• Sans ORDER BY, LIMIT 1 prend un groupe au hasard : toujours trier avant de limiter.

• Pour trouver le minimum, WHERE age = (SELECT MIN(age) ...) est plus fiable que ORDER BY ... LIMIT 1 : il affiche aussi les égalités.

• Vérifier le nom exact des colonnes avec DESCRIBE nom_table; en cas d'erreur Unknown column.

• Tester chaque CTE seul, étape par étape, est la meilleure façon de comprendre une requête complexe.

• La moyenne des dépenses vaut 358.75 si on ne compte que les vrais clients, et 311 si on inclut la commande orpheline du client 9 : signaler ce genre de choix.

8. Prochaine séance

Les fonctions de fenêtrage (ROW_NUMBER, RANK, SUM OVER) : calculer sur un ensemble de lignes sans les regrouper.